

Library Management System

Java GUI Desktop Application | OOP Design, SQLite JDBC, Multithreading & DAO Architecture

Project Presentation & Architecture Overview

1. Executive Project Summary

Key System Features

- **Complete CRUD Inventory:** Books & E-Books management with custom properties.
- **User Directory:** Student & Faculty profile registration with role-based policies.
- **Issue & Return Processing:** Real-time book issue, due-date calculation, and fine handling.
- **Multithreading Console:** Asynchronous background fine calculation sweeps & catalog CSV backup exports.

Technical Architecture

- **GUI Framework:** Java Swing (EDT event dispatching, JTabbedPane, JTable, RowSorter)
- **Database:** SQLite JDBC Driver (Zero configuration embedded DB)
- **Design Patterns:** Singleton (DatabaseManager), Generic DAO Repository Pattern, Model-View-Service Separation

2. Object-Oriented Programming (OOP)

Implementation

Comprehensive compliance with course OOP requirements through core concepts:

Concept	Implementation Class / File	Architectural Purpose
Interfaces	Borrowable, Searchable, GenericDAO	Establishes behavioral contracts across entities and repository layers.
Inheritance	LibraryItem → (Book, EBook) User → (StudentUser, FacultyUser)	Base abstract classes define common attributes while subclasses specialize properties.
Polymorphism	calculateOverdueFine(), calculateDiscountedFine()	Physical books (\$1.50/day) vs EBooks (\$0.50/day); Faculty enjoy a 50% fine discount.
Custom Exceptions	BookNotAvailableException, UserLimitExceededException	Encapsulates domain error rules with clear, user-friendly messages.

3. Database Connectivity (JDBC) & DAO Architecture

DAO Repository Design

Data Access Object (DAO) classes separate SQL query mechanics from business logic:

- BookDAO: Maps JDBC ResultSet to Book and EBook domain models.
- UserDAO: Persists and fetches StudentUser and FacultyUser records.
- TransactionDAO: Manages issue/return records and fine status.

PreparedStatement & Security

```
String sql = "INSERT INTO items\nVALUES (?, ?, ?, ?, ?, ?, ?, ?)";\ntry (PreparedStatement pstmt =\n    conn.prepareStatement(sql)) {\n    pstmt.setString(1,\n        item.getItemId());\n    pstmt.setString(2, item.getTitle());\n    // Safe parameter binding prevents\n    // SQL Injection\n    pstmt.executeUpdate();\n}
```

4. Multithreading & Synchronization

Concurrent thread execution for background tasks and thread safety:

Background Worker Tasks

- **Runnable Thread** (`FineCalculationTask`): Periodically audits active loans, computes overdue charges, and updates database records asynchronously.
- **Callable Thread** (`BackupTask`): Exports catalog data to CSV format using thread pool futures without freezing the UI.

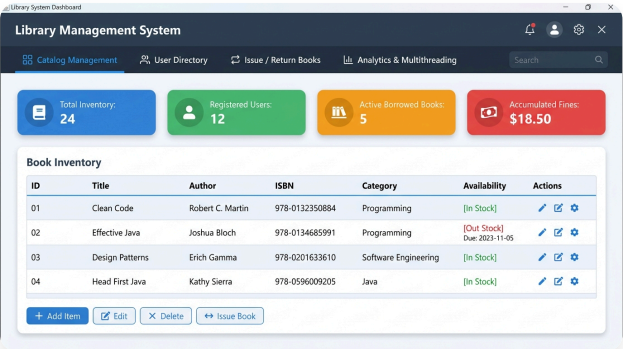
Thread Synchronization & Safety

Uses `ReentrantLock` in `LibraryService` to prevent double-booking race conditions during simultaneous book issue requests:

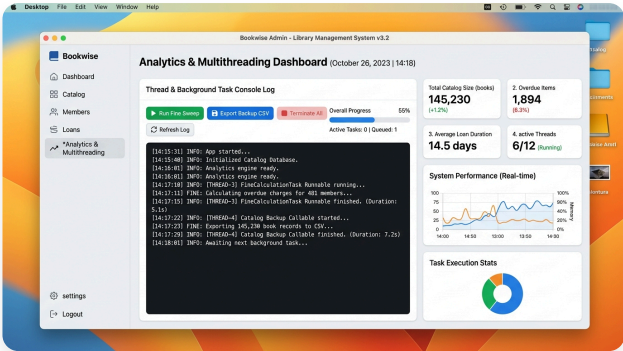
```
private final ReentrantLock  
transactionLock = new  
ReentrantLock(); public Transaction  
issueBook(String uid, String iid) {  
    transactionLock.lock(); try { //  
        Safe concurrent inventory check &  
        issue } finally {  
        transactionLock.unlock(); } }
```

5. Application Screenshots & User Interface

Catalog & Inventory Dashboard



Analytics & Multithreading Console



6. Evaluation Rubric Compliance Matrix

Rubric Evaluation Category	Implementation Highlights	Evidence Code Files
OOP Implementation (Polymorphism, Inheritance, Exceptions, Interfaces)	Interfaces, Abstract Classes, Method Overriding, Domain Exception Hierarchy	LibraryItem, Book, EBook, User, StudentUser, FacultyUser, Borrowable, LibraryException
Collections & Generics	Generic Data Access Object Interface, List queries, Map in-memory caching	GenericDAO, List, Map cache
Multithreading & Synchronization	Runnable & Callable worker threads, ReentrantLock concurrency control	FineCalculationTask (Runnable), BackupTask (Callable), ReentrantLock
Classes for Database Operations (DAO)	Decoupled Repository Pattern classes for items, users, and transactions	BookDAO, UserDAO, TransactionDAO
Database Connectivity (JDBC)	SQLite Connection Manager, PreparedStatement, ResultSet mapping	DatabaseManager (SQLite connection pool & PreparedStatement handling)